ECE 260C, Spring 2026

Routing: GRT and CTS

Andrew B. Kahng

Global Routing Problem

- Input:
  - Placed netlist
  - Technology info (routing pitches, design rules, “tech LEF”, etc.)
  - Design information (chip size, number of routing layers, etc.)
  - Obstacles and pre-routes
- Output:
  - Assignment of each net to particular routing regions
- Objective:
  - Overflow / congestion (for improving the completion rate of the detailed routing)
  - Total wirelength
  - Total number of vias
  - Timing, crosstalk, etc.

Graph Models for Global Routing

- Global routing is a **graph problem**
- Model routing regions, and their adjacencies and capacities, as graph vertices, edges and weights
- Choice of model depends on algorithm
- Grid graph model
  - Grid graph represents layout as a $h \times w$ array
  - Vertices are layout cells
  - Edges capture cell adjacencies
  - Zero-capacity edges represent blocked cells

Courtesy K. Keutzer et al. UCB

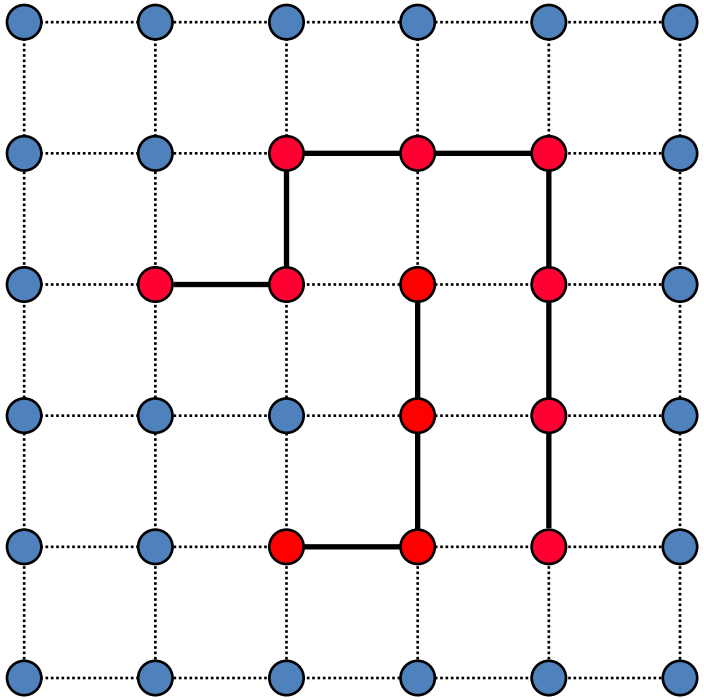
Key Concepts Today

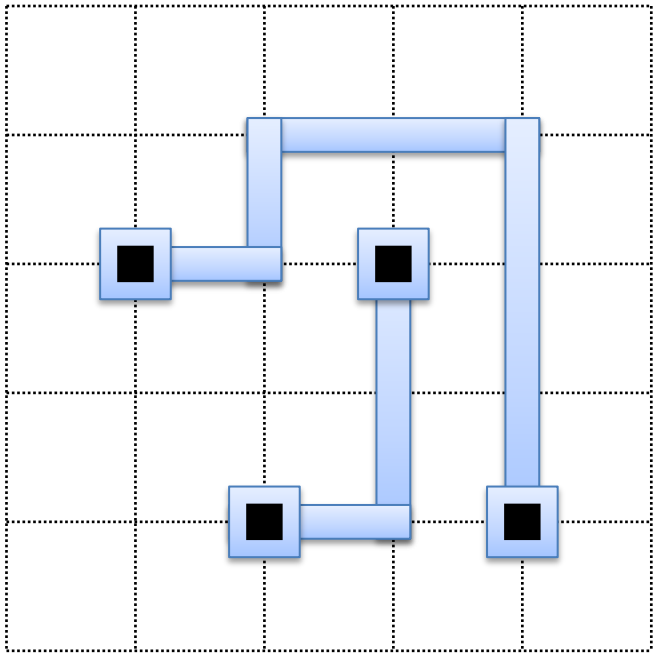
https://github.com/The-OpenROAD-Project/TritonRoute
https://github.com/ABKGroup/TritonRoute-WXL
(see “drt” and “grt” in OpenROAD /src)

- **Routing one net: connection**
  - 2-pin connection (Dijkstra, A*)
  - Multi-pin
- **Routing multiple nets: compatibility**
  - Ordering (Sequential routing)
  - Ripup-reroute
- **Orchestration: → a routing tool**
  - Coarse-to-fine grain management of complexity: **global**, **detailed**
    - 2D vs. 3D, Gcell size, Sbox (“worker”) size, Search-and-Repair
  - Separable subproblems
    - Crosspoint Assignment, Sbox (“worker”) for detailed route, Steiner edges for global route
  - Costing (vias, overflows, timing, crosstalk, yield, …) and Re-Costing
  - Use scenarios
    - Incremental (ECO), special (clock, power), non-default routing, …

Foundational Routing Structure: Grid Graph

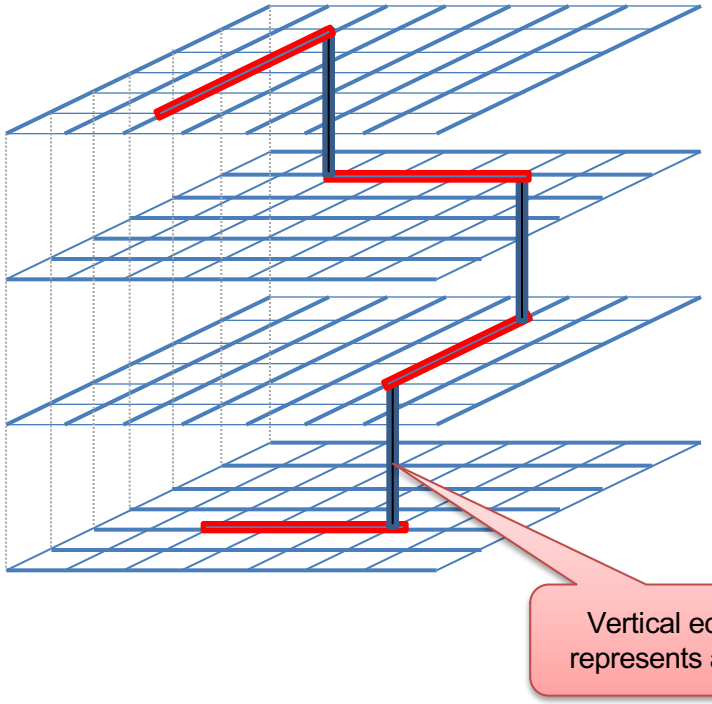
- Regular grid graph:
  - Edges represent short (legal) wire segments
- Nodes are marked:
  - Free (blue)
  - Occupied (red)
- Often other features are encoded
  - Id of the net, pin, etc.

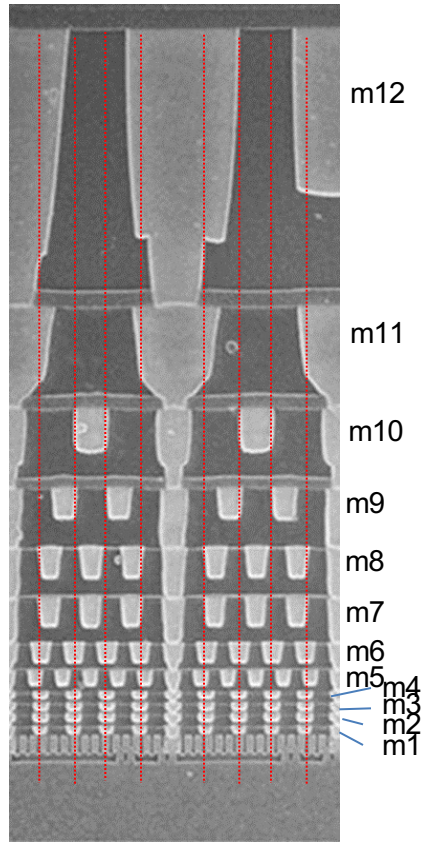




Multi-Layer Routing on a Grid Graph

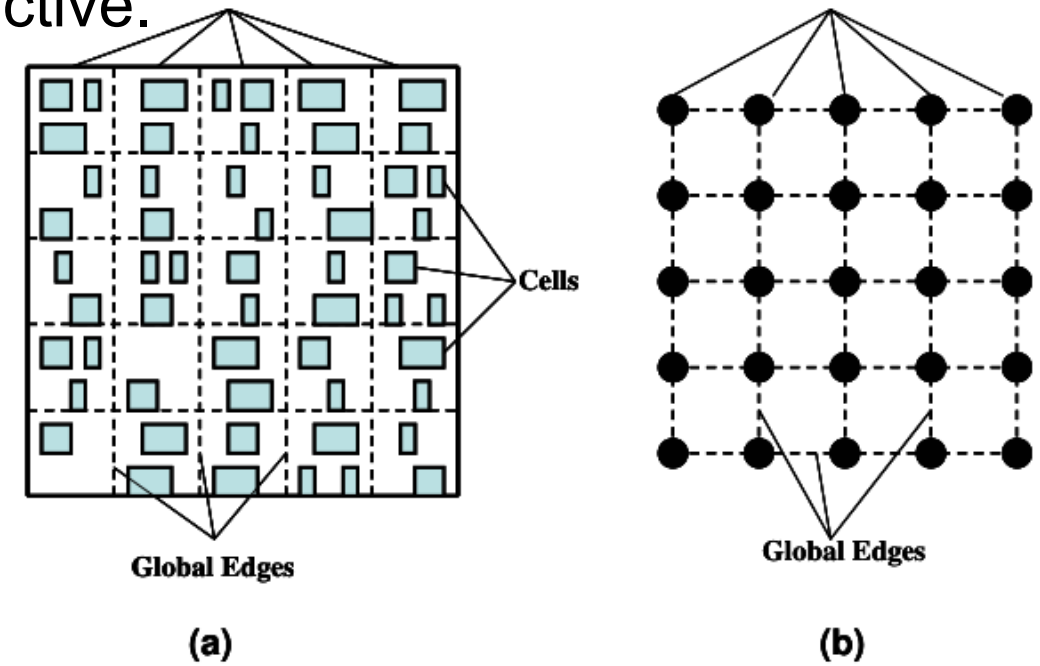
- 2D routes are assigned layers. Then, refinement is done over the 3D graph
- Example of the first 4 layers as a grid graph





Grid Graph Capacity Model in GRT

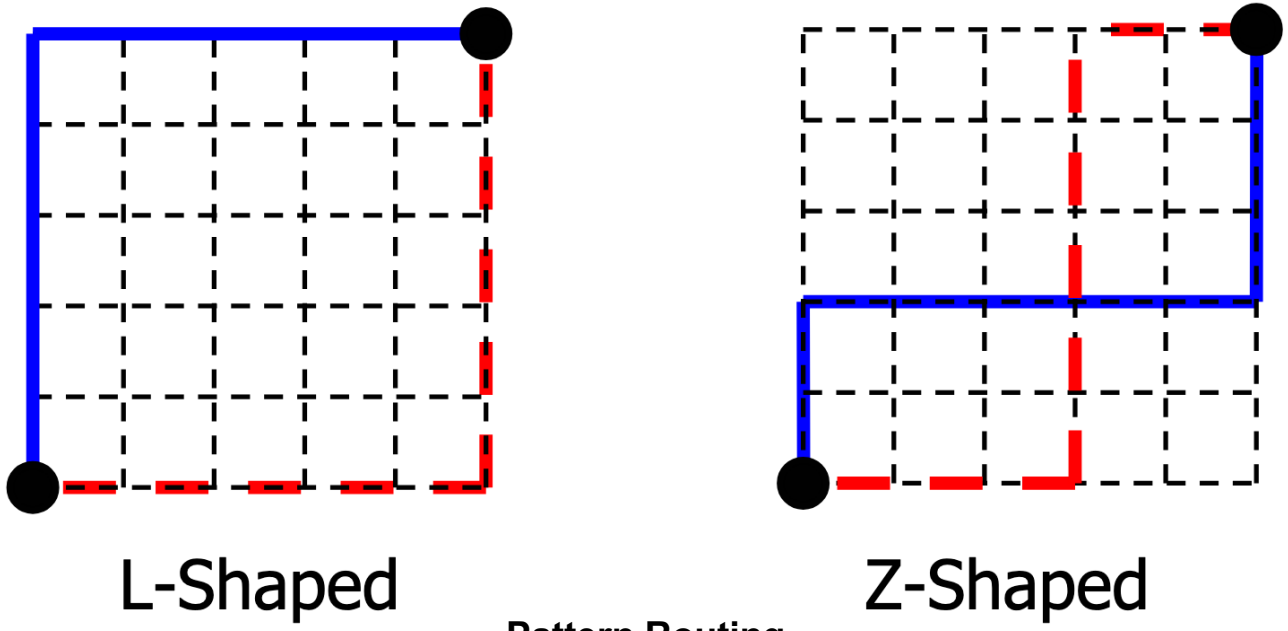
- GRT abstracts detailed geometry into GCells
- Each edge carries capacity, usage, and overflow
  - ∑overflow is the primary routability objective.
- Capacity based on:
  - Number of tracks and layers,
  - Track preferred directions,
  - Blockages,
  - And user-adjustable derates.



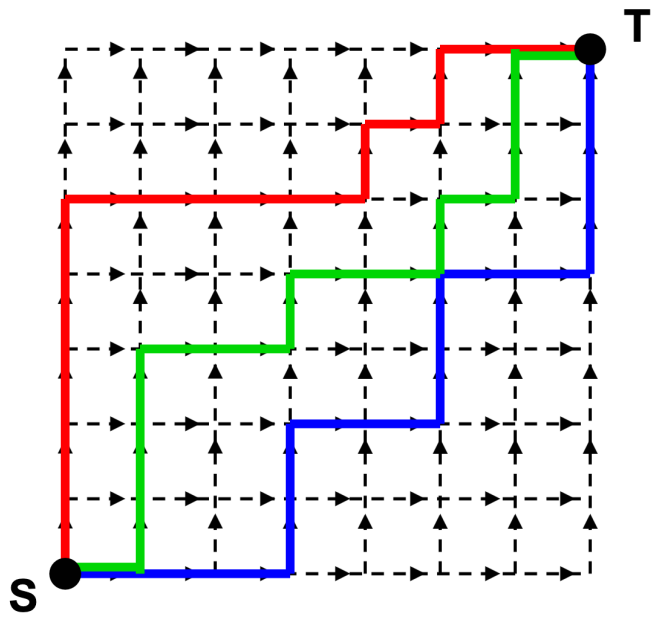
- 2D-first approach
  - Per-layer 3D resources are summed into the 2D accounting used by the primary global-routing loop

Pattern and Monotonic Routing

- Global routing scales by using fast initial pattern routing
  - L/Z shapes, three-bend patterns
- Monotonic routing provides a deterministic set of routes
- Loop of: select pattern, test congestion/blockages, select best
  - Then, apply repair (ripup-and-reroute)



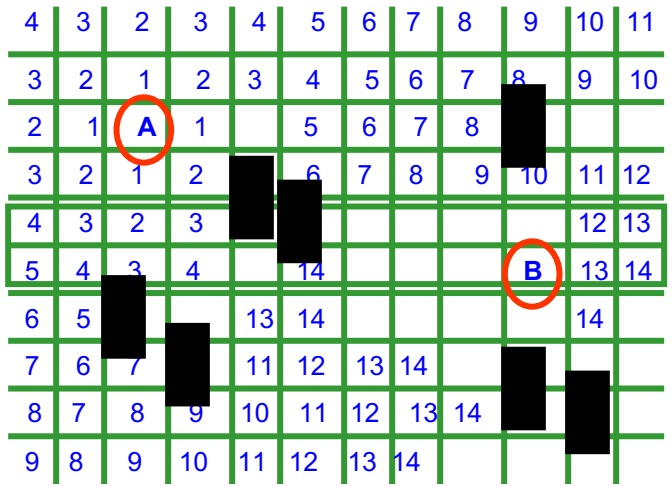
Pattern Routing



Monotonic Routing

Maze Routing

- Point to point routing of nets
  - from source to sink
- Basic idea = wave propagation (Lee, 1961)
  - Breadth-first search + back-tracing after finding shortest path
  - Guarantees to find the shortest path
- Objective = route all nets according to some cost function that minimizes congestion, route length, coupling, etc.

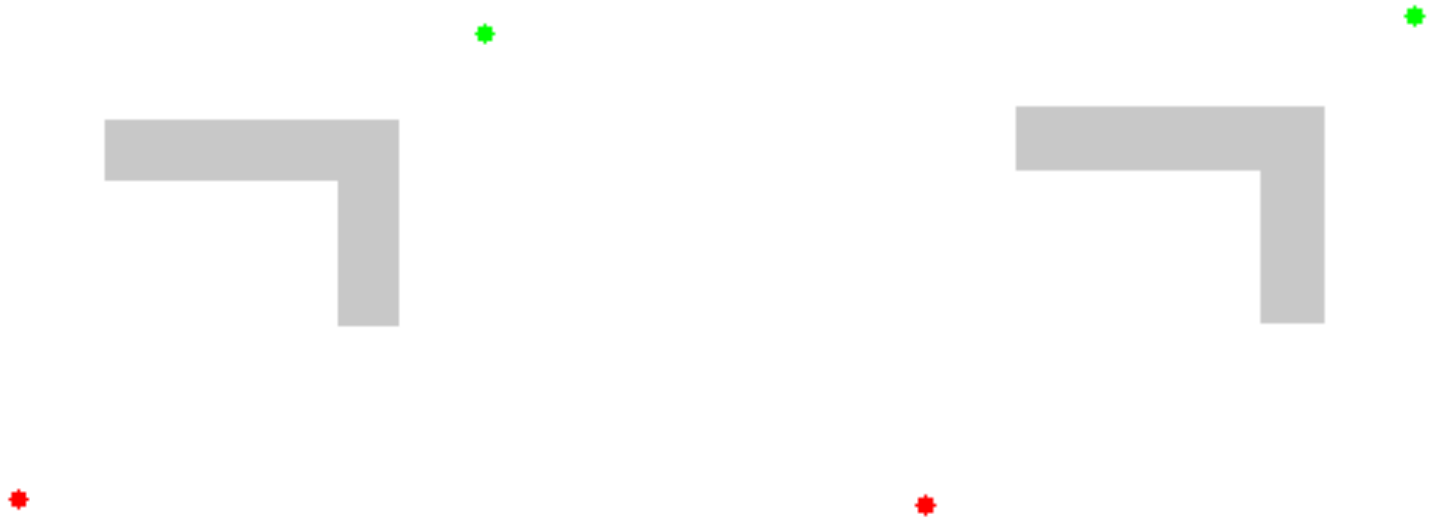


Cost = label
= distance from “A”

Courtesy K. Keutzer et al. UCB

Finding Connections: Dijkstra vs. A*

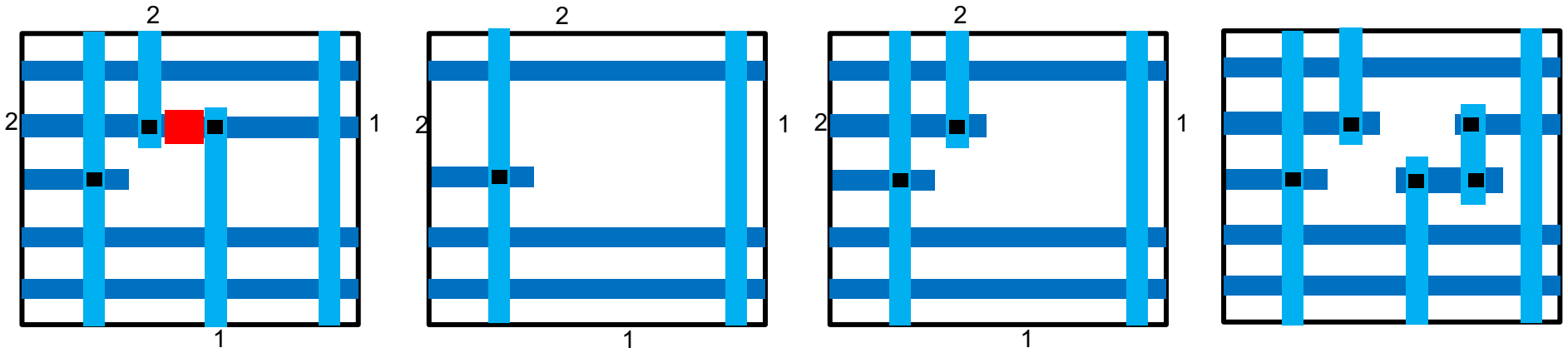
Dijkstra ("BFS with Alarm Clocks") vs. A* ("Best-First Search")



Source: Wikipedia

- Even with obstacles present, given an optimistic lower bound h(n) on "future cost", A* (aka "best-first search") will guarantee to find a lowest-cost path from Source to Target
- Speed on a grid graph with n x n nodes: O(n) best case, $O(n^2)$ worst case

Incremental Ripup and Reroute



- Small "switchbox" (in DRT)
  - Detailed routing problem is typically decomposed into switchboxes, with solutions then stitched back together …
  - GRT problem is over (sets of) Steiner tree (edges)
- Routed sequentially, net by net (Net1 before Net2 → spacing violation)
- In GRT, we see congestion/routability violations that force ripup-reroute

Rip up offending nets

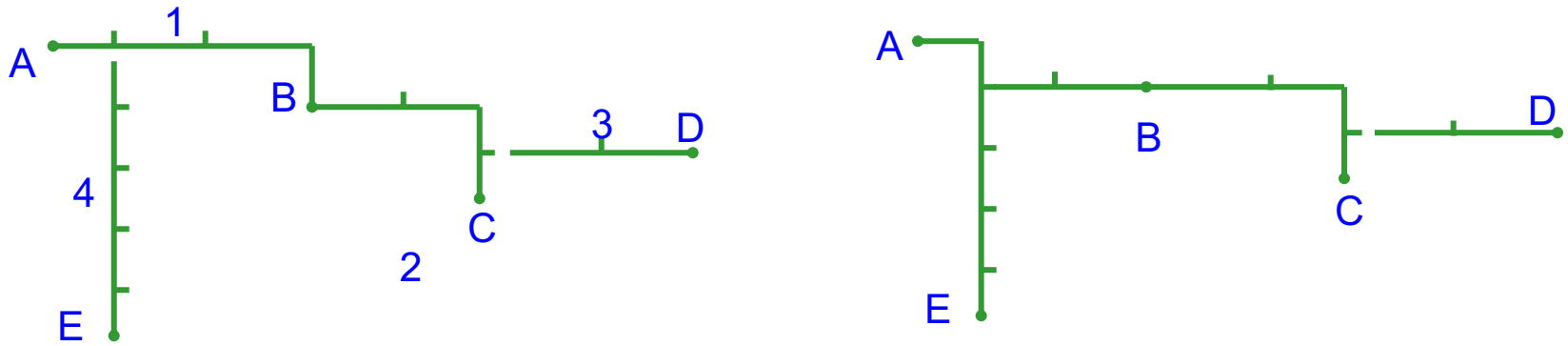
Route Net2

Then route Net1

Note: this was simply a lucky guess

Connecting Multi-Terminal Nets

- In general, maze and line-probe routing are not well-suited to multi-terminal nets
- Several attempts made to extend to multi-terminal nets
  - Connect one terminal at a time
  - Use the entire connected subtrees as sources or targets during expansion
- Improve solution quality
  - Ripup/Reroute to improve solution quality (remove a segment and re-connect)
  - In GR: some topological perturbation (edge shifting)

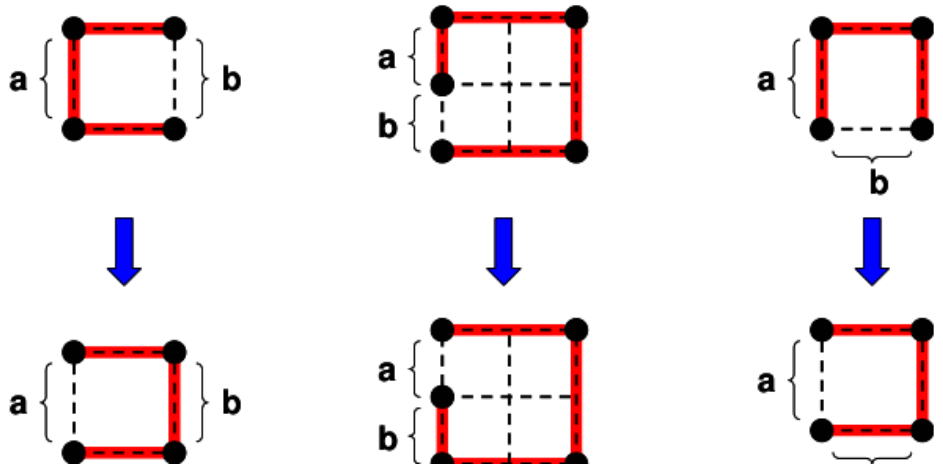


- Results are sub-optimal
- Inherit time and memory cost of maze and line-probe algorithms

Courtesy K. Keutzer et al. UCB

Topology Selection

- Multi-pin nets must be decomposed before two-pin routing: topology is the first routing decision for each net.
- For a given pin set: the chosen tree affects the balance of congestion, wirelength, via count, and timing
- OpenROAD's `stt` implements fast FLUTE RSMTs for minimizing wirelength.
  - Routability and timing need more

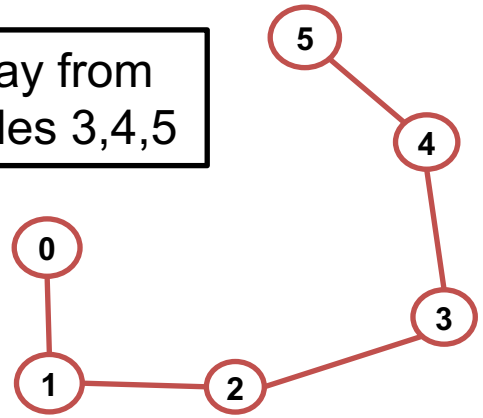


Changing topology moves routing demand before maze routing starts, even though the pin set is unchanged.

Prim-Dijkstra Construction

- Timing-aware trees trade off source-sink radius against total wirelength.

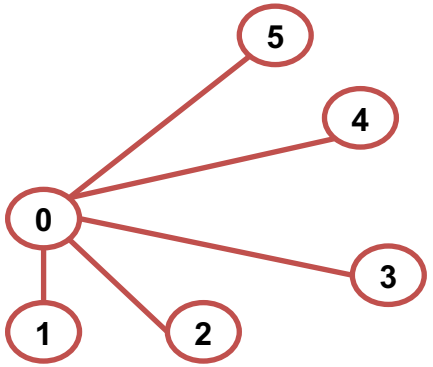
But large delay from source for nodes 3,4,5



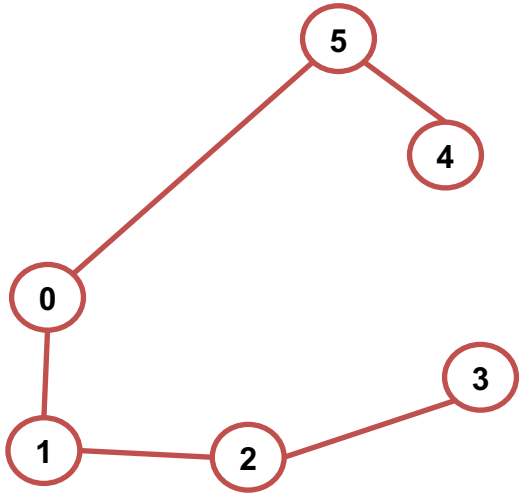
Prim's Minimum Spanning Tree (MST)
Minimizes wirelength (WL)

Dijkstra's Shortest Path Tree (SPT)
Minimizes source-sink pathlengths (PLs)

But large tree wirelength!



Prim-Dijkstra (PD) tradeoff of Prim's MST and Dijkstra's SPT



Directly trades off the Prim, Dijkstra constructions

Prim-Dijkstra Gives a Timing-Aware Tree Knob

- MST-like trees minimize total wirelength; SPT-like trees minimize source-to-sink pathlength.
- Prim-Dijkstra blends the two with α, letting topology trade routing cost against timing radius.
- This matters because minimum-wirelength trees can create large detours, while shortest-path trees can waste routing resources.
- The idea is simple enough to implement greedily and useful as intuition for timing-aware topology selection.

```
Prim-Dijkstra core

Choose edge e_ij minimizing
    d_ij + α · l_i

α = 0  -> Prim-like low-WL tree
α = 1  -> Dijkstra-like low-radius tree

Greedy update:
  add (i,j) to T
  set l_j = l_i + d_ij
```

https://vlsicad.ucsd.edu/Publications/Journals/j18.pdf
https://vlsicad.ucsd.edu/Publications/Conferences/355/c355.pdf

Capacity Control: Global Margin and Local Steering

- Layer adjustment globally scales the capacity available to GRT, creating detailed-routing margin
- Macro extension reserves a halo region around macros
- Region adjustment changes capacity inside a chosen rectangle to steer demand away from identified potential hostpots

```
Capacity controls

Global margin:
  set_global_routing_layer_adjustment ...

Macro halo:
  set_macro_extension 2

Local hotspot steering:
  set_global_routing_region_adjustment ...
```

Derate Example: ASAP7

- ORFS ASAP7 enablement deliberately scales effective routing capacity **down by 25%**.
  - This spreads demand earlier and helps avoid detailed routing getting stuck on overly tight guides.
- Pessimism acknowledges advanced-node pin access, via limits, and DRC effects that the global graph cannot model effectively.

```
ASAP7 fastroute.tcl

set_global_routing_layer_adjustment \
  $::env(MIN_ROUTING_LAYER)-
$::env(MAX_ROUTING_LAYER) 0.25

set_routing_layers -clock \
  $::env(MIN_CLK_ROUTING_LAYER)-
$::env(MAX_ROUTING_LAYER)

set_routing_layers -signal \
  $::env(MIN_ROUTING_LAYER)-
$::env(MAX_ROUTING_LAYER)
```

Challenges for Global Router Development

- Runtime and memory scalability
- Congestion prediction
  - Congestion-aware global routing?
  - History-based cost functions (NCR)?
- Net routing order
  - How to maximize the concurrency of routing decisions?
  - ILP or multi-commodity flow based methods?
  - Rip-up and reroute (or Reroute w/o Ripup)?

- Steiner tree topology reconstruction
  - Steiner point relocation
  - Complete solution space exploration
  - RSMT or RMST?

Tool users have opportunities to guide the global router – weights, priorities, non-default rules, layer ranges, density screens, track supply reductions, … See, e.g., https://vlsicad.ucsd.edu/Publications/Conferences/411/c411.pdf

- Timing, placement-synthesis services, fidelity to detailed routing, …

GRT Commands and Knobs

- `set_routing_layers` and layer adjustment define where traffic may go and how much capacity GRT should believe.
- `set_macro_extension` and region-level adjustments steer traffic away from local hotspots before DRT fails.
- `-congestion_iterations`, `-critical_nets_percentage`, and `-skip_large_fanout_nets` trade quality, timing focus, and runtime.
- `-allow_congestion`, incremental reroute flags, and experimental hooks expose triage or specialized behavior.

```
global_route -guide_file route.guide \
  -congestion_iterations 50 \
  -critical_nets_percentage 30 \
  -congestion_report_file congestion.rpt

set_routing_layers -signal M2-M10 -clock M6-M9
set_global_routing_layer_adjustment M4-M8 0.30
set_macro_extension 2
```

When GRT Fails, Match the Symptom to the Knob

- Global overflow everywhere usually means placement is too dense: reduce utilization or add whitespace before over-tuning the router.
- Layer-specific or macro-edge hotspots point to layer adjustment, layer ranges, macro extension, or region adjustment.
- If guides look clean but DRT still fails, investigate pin access or local geometric issues rather than just global congestion.
  - `draw_route_segments {my_net}`
- Near-converged runs may need more congestion iterations; noisy high-fanout nets may need to be skipped for debug.

CTS

Clock Distribution

- General goal of clock distribution
  - Deliver clock to all memory elements with acceptable skew
  - Deliver clock edges with acceptable sharpness
- Clocking network design is one of the greatest challenges in the design of a large chip
- Clocks generally distributed via wiring trees (and meshes)
- Low-resistance interconnect to minimize delay, variability
- Low-capacitance interconnect to minimize power
- Multiple drivers to distribute clock signal
  - Use optimal sizing principles to design buffers
  - Clock lines can create significant crosstalk

Issues in Clock Distribution Network Design

- Skew
  - Process, voltage, and temperature (especially across “wide corners” – 1.25V to 0.70V, 125C to -40C, etc.) https://vlsicad.ucsd.edu/Publications/Conferences/327/c327.pdf
  - Data dependence
  - Noise coupling
  - Load balancing
- Power, CV^2f
  - Clock gating
- Flexibility/Tunability
  - Compactness – fit into existing layout/design
- Reliability
  - Electromigration

OpenROAD CTS Commands

Command	Description	Example Output

`clock_tree_synthesis`	Build a balanced Htree by choosing appropriate clock buffers	`[INFO CTS-0050] Root buffer is BUF_X4.`
		`[INFO CTS-0051] Sink buffer is BUF_X4.`
		`[INFO CTS-0052] The following clock buffers will be used for CTS:`
		`BUF_X4`
		`. . .`
		`[INFO CTS-0017] Max level of the clock tree: 5.`
		`[INFO CTS-0098] Clock net "clk"`
		`[INFO CTS-0099] Sinks 2537`
		`[INFO CTS-0100] Leaf buffers 96`
		`[INFO CTS-0101] Average sink wire length 9247.25 um`
		`[INFO CTS-0102] Path depth 18 - 19`
		`[INFO CTS-0207] Leaf load cells 62`
		`[INFO RSZ-0058] Using max wire length 693um.`
		`[INFO RSZ-0047] Found 33 long wires.`
		`[INFO RSZ-0048] Inserted 94 buffers in 33 nets.`

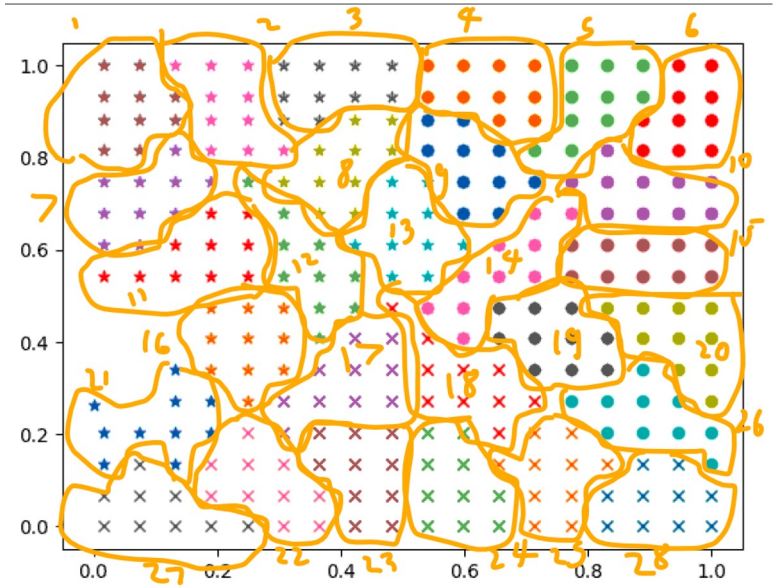
`configure_cts_characterization`	`-max_slew <ps> -max_cap <fF> -slew_steps <N> -cap_steps <N>` Set maximum values for characterization to test against. Set number of discrete buckets "steps" that slew/cap values will be divided into.	`N/A`
`repair_clock_nets`	Fixes LDRC violations including max wire length	`[INFO RSZ-0058] Using max wire length 2154um.`

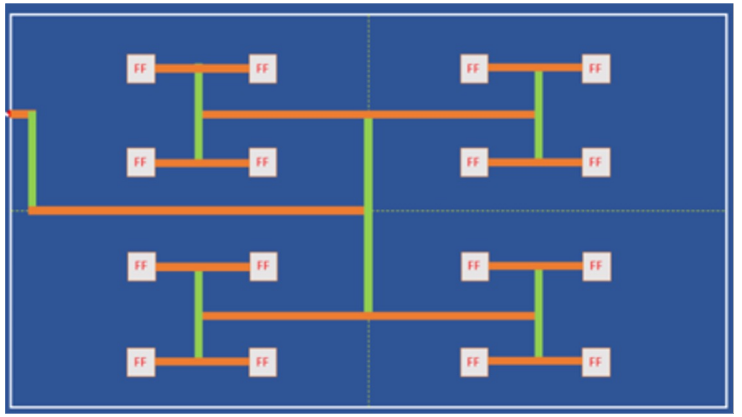
`report_clock_skew`	Report worst clock skew for each clock signal in the design	`Clock clk`
		`1.26 source latency inst_7_12/clk ^`
		`-1.13 target latency inst_8_12/clk ^`
		`0.00 CRPR`
		`--------------`
		`0.13 setup skew`

`report_checks -format full_clock_expanded`	Report timing violations including clock paths	`Startpoint: dp.rf.rf[31][3]$_DFFE_PP_`
		`(rising edge-triggered flip-flop clocked by clk)`
		`Endpoint: aluout[0] (output port clocked by clk)`
		`Path Group: clk`
		`Path Type: max`
		
		`Fanout Cap Slew Delay Time Description`
		`---------------------------------------------------------------------------`
		`0.00 0.00 clock clk (rise edge)`
		`0.00 0.00 clock source latency`
		`1 0.09 0.00 0.00 0.00 ^ clk (in)`
		`clk (net)`
		`0.00 0.00 0.00 ^ clkbuf_0_clk/A`
		`(sky130_fd_sc_hd__clkbuf_16)`
		`8 0.21 0.22 0.25 0.25 ^ clkbuf_0_clk/X`
		`(sky130_fd_sc_hd__clkbuf_16)`

CTS Main Steps

- Sink clustering
  - Sequential elements are grouped into a fixed number of clusters based on their locations
- Tree construction and balancing
  - Buffers are inserted based on some structure, e.g., hierarchical H-Tree
  - Tree lengths are balanced such that clock skews are minimized
- LDRC (“ERC”, “DRV”) **electrical** rules repair
  - LDRC violations are repaired during or after CTS
    - Max transition, max capacitance, max wire length, etc.

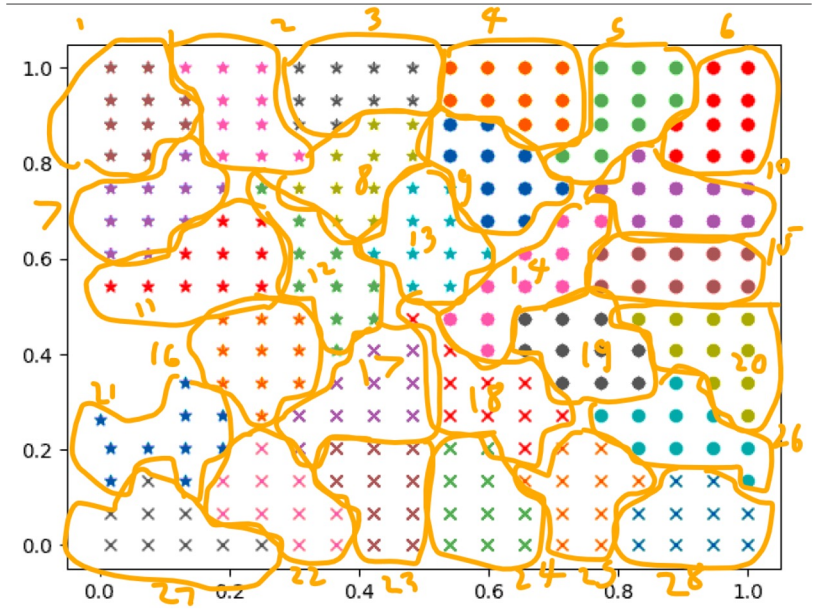




Thanks: Dr. Cho Moon, Precision Innovations

Sink Clustering

- Group sequential elements based on their locations to produce the best results (e.g., minimum wire length)
  - These parameters can be specified manually or determined automatically
    - Cluster size
    - Cluster diameter
- All elements in the cluster will be driven by the same buffer



Early “TritonCTS” versions used spacefilling curves to perform sink clustering !

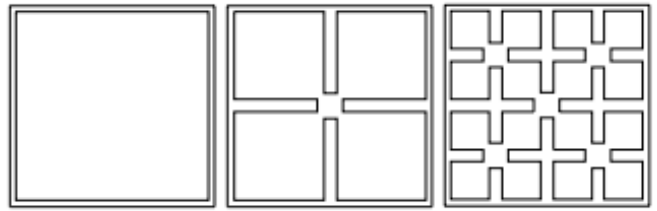


Figure 2: The planar Sierpinski spacefilling curve.

https://vlsicad.ucsd.edu/Publications/Conferences/32/c32.pdf

Thanks: Dr. Cho Moon, Precision Innovations

Sink Clustering Implementation

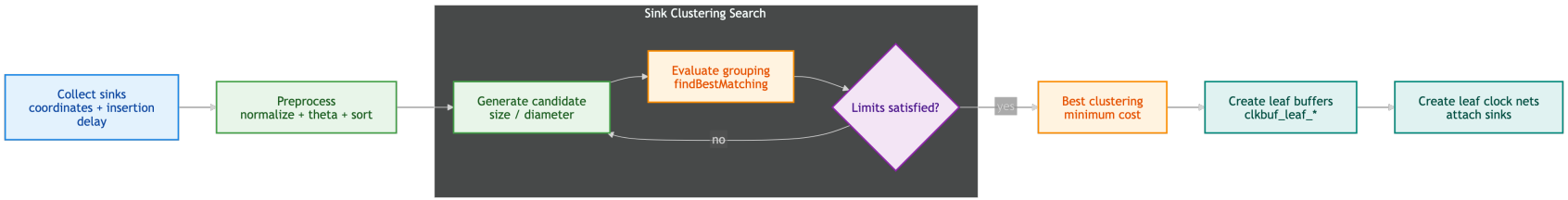
- `normalizePoints` maps sinks into a unit box
- `computeTheta` recursively computes a locality-preserving order key θ
- `sortPoints` orders sinks by theta

```
normalizePoints(maxDiameter);
computeAllThetas();
sortPoints();
bestSolutionFound =
        findBestMatching(groupSize);

limit:  size >= groupSize
        or diameter > maxInternalDiameter
        or cap > sinkBufferInputCap * factor
```

- `findBestMatching` tests groupings and tracks best cost

- `isLimitExceeded` enforces size, diameter, or cap-based limits

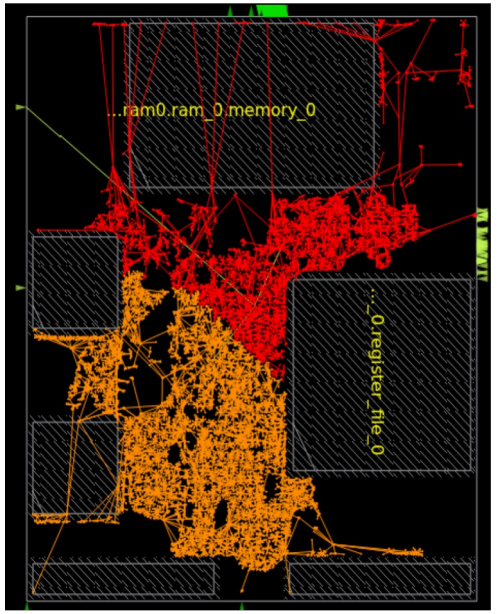


Sink Clustering Flow

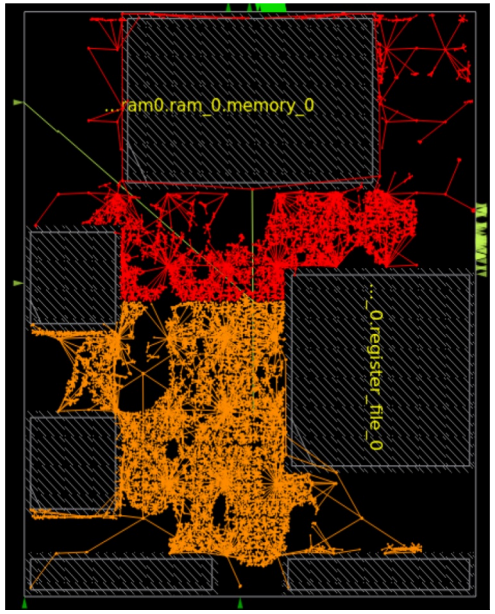
- Collect sink coordinates and insertion-delay data
- Run `SinkClustering` over candidate size/diameter combinations
  - Choose the best clustering solution found
- Create leaf buffers such as `clkbuf_leaf_*`
- Create leaf clock nets and attach clustered sinks

Obstruction-Aware CTS

- Clock buffers should not be placed on top of macros, placement blockages or another clock buffers
  - Detailed placement may displace “illegal” buffers and cause timing to change after CTS
  - New “legal” buffer locations need to preserve balanced clock tree
  - Obstruction-aware CTS can reduce legalizer displacement by up to 4X



sky130hd/microwatt **without** obstruction-aware CTS



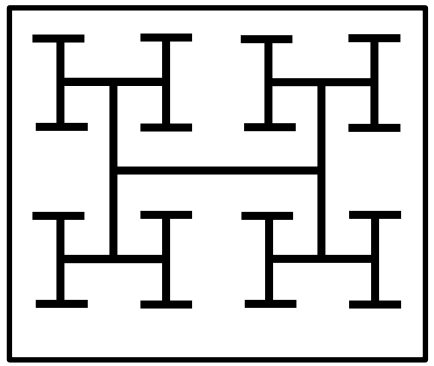
sky130hd/microwatt **with** obstruction-aware CTS

Thanks: Dr. Cho Moon, Precision Innovations

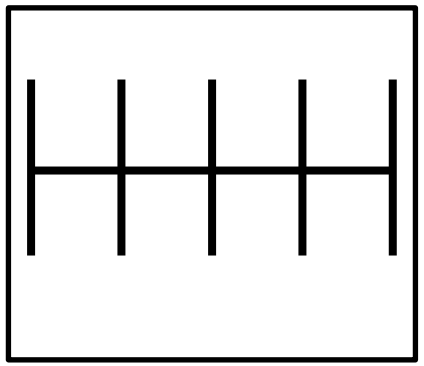
CTS Origin: Generalized H-Tree Concept

- Structured clock trees

K. Han, A. B. Kahng and J. Li, "Optimal Generalized H-Tree Topology and Buffering for High-Performance and Low-Power Clock Distribution", *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* **https://vlsicad.ucsd.edu/Publications/Journals/j128.pdf**.



H-tree



"Fishbone"

	H-tree	Fishbone
Skew	<	
Wirelength	>	
Latency	>	
Power	>	

Generalized H-tree (GH-tree)

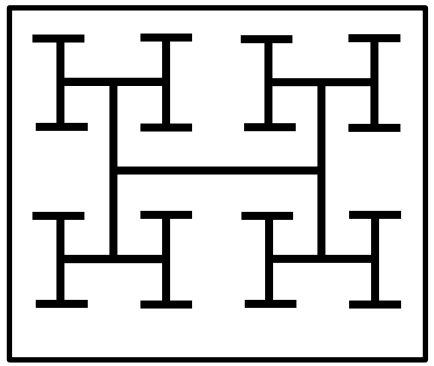
Can we mix two clock structures to have better tradeoff between clock power vs. skew or latency?

- History: (1) Bakoglu's 1988 book made H-tree approach well-known. Cadence CTGen (Dr. Lars Hagen), mid-1990s, started trend toward "fishbone" style – save capacitance!
- Starting in the 2010s: on-chip variation (OCV) derates became very costly, so goal became to reduce insertion delay (== "latency") along with power.

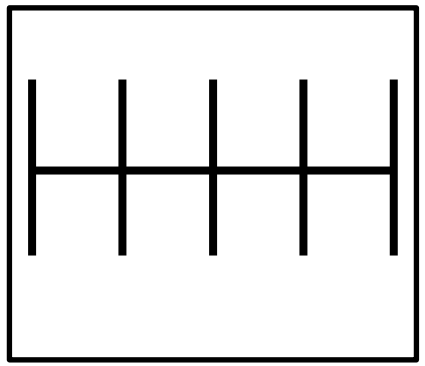
Generalized H-Tree Concept

- Structured clock trees

K. Han, A. B. Kahng and J. Li, "Optimal Generalized H-Tree Topology and Buffering for High-Performance and Low-Power Clock Distribution", *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* https://vlsicad.ucsd.edu/Publications/Journals/j128.pdf.



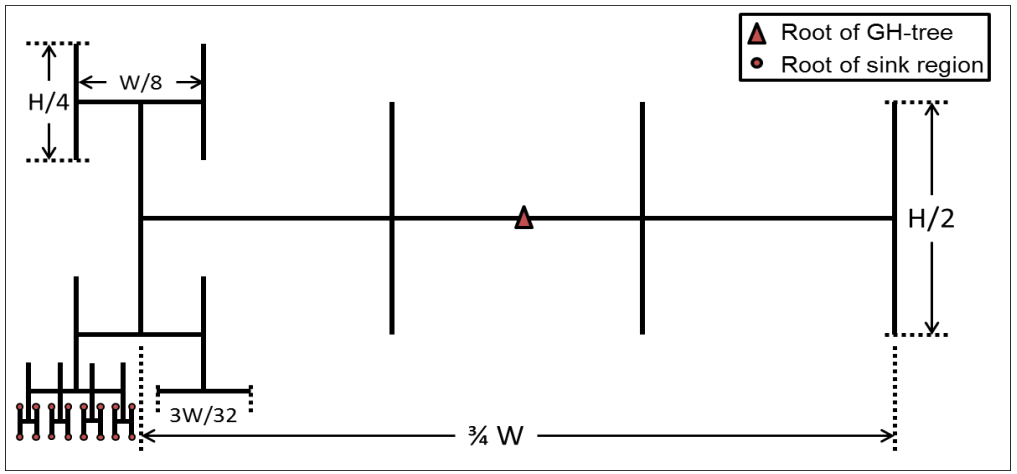
H-tree



"Fishbone"

	H-tree	Fishbone
Skew	<	
Wirelength	>	
Latency	>	
Power	>	

Generalized H-tree (GH-tree)

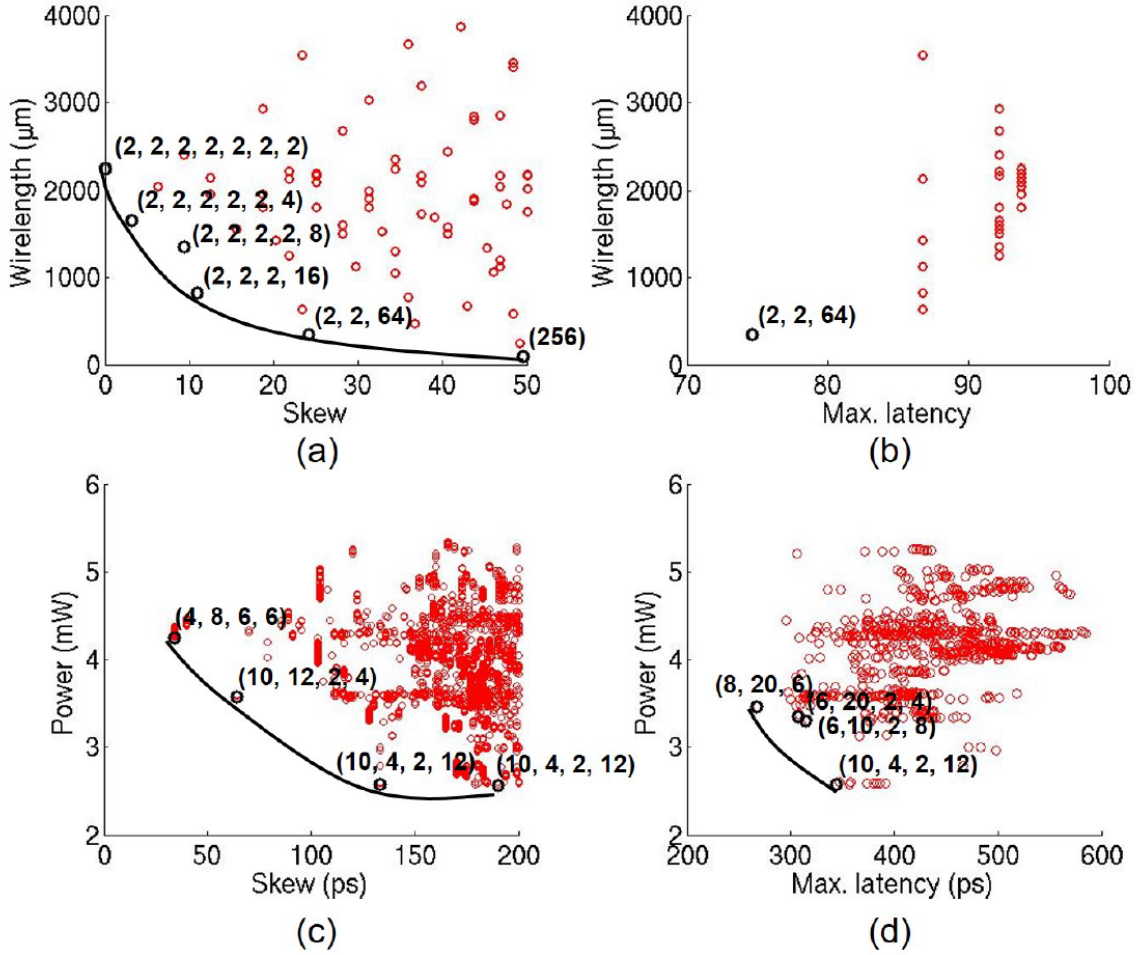


a balanced tree topology with arbitrary branching factor at each level

GH-tree with depth P = 8 and branching factors (4, 2, 2, 2, 4, 2, 2, 2)

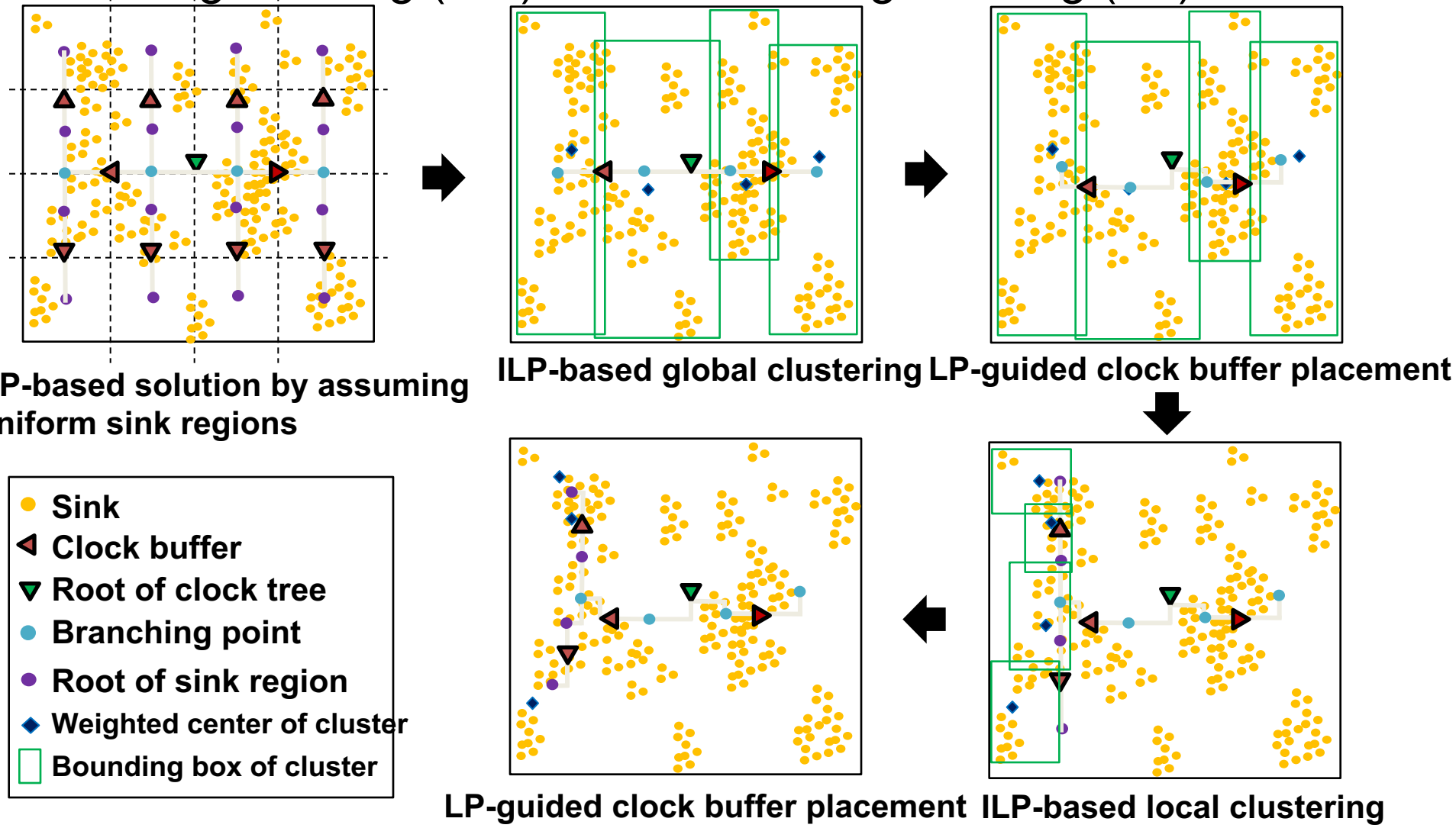
Idea: Capture/Explore Tradeoff ("Pareto" Frontier)

- **(Recall: floorplan shape functions ?)**
- GH-tree explores branching patterns across levels
- Power, skew, latency, and wirelength create a Pareto tradeoff
- Buffering and topology must be co-optimized

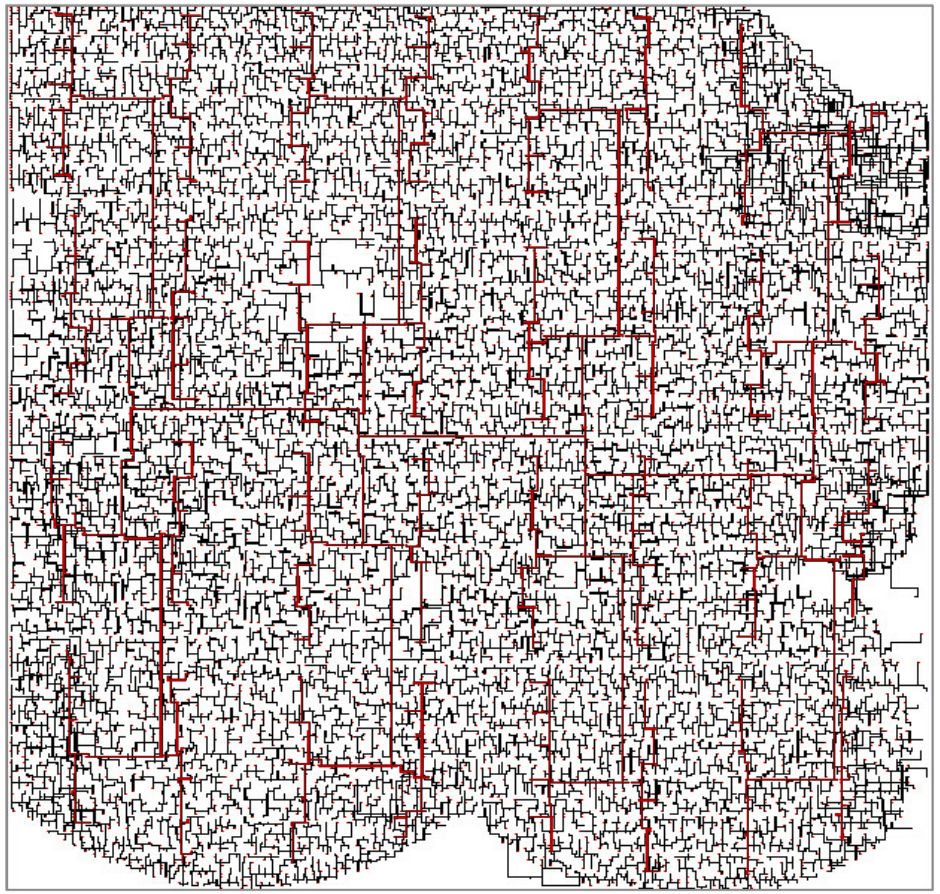


Embedding of GH-Tree Into a Sink Placement

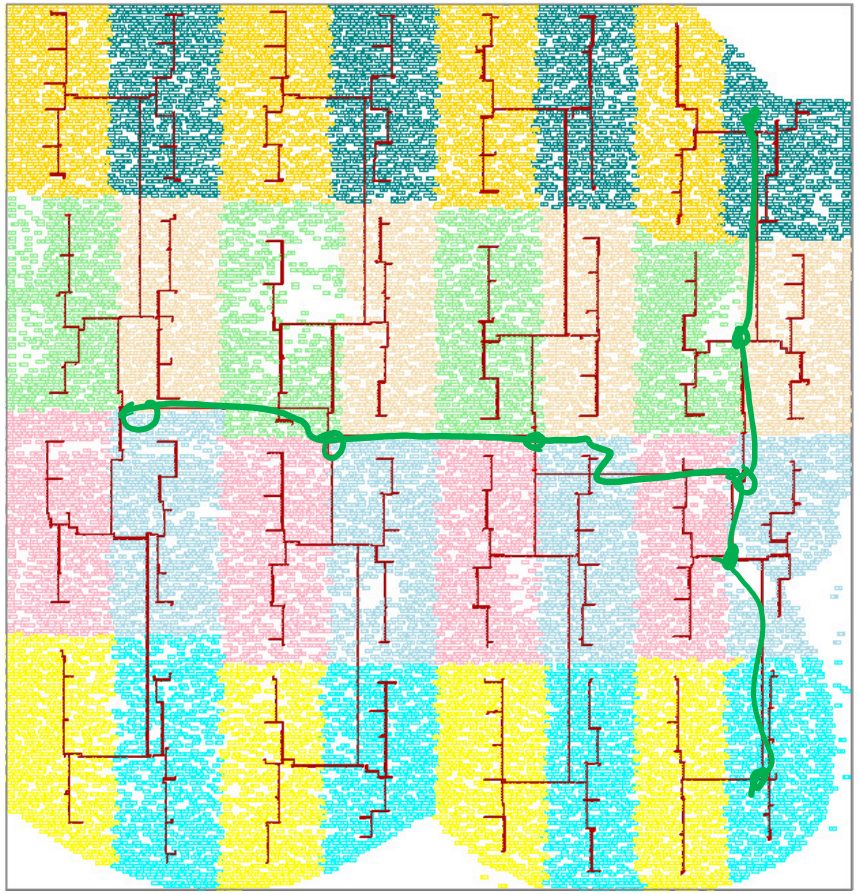
- Clustering and branching point / buffer placement using Integer Linear Programming (ILP) and Linear Programming (LP)



Layout Example of GH-tree on VGA



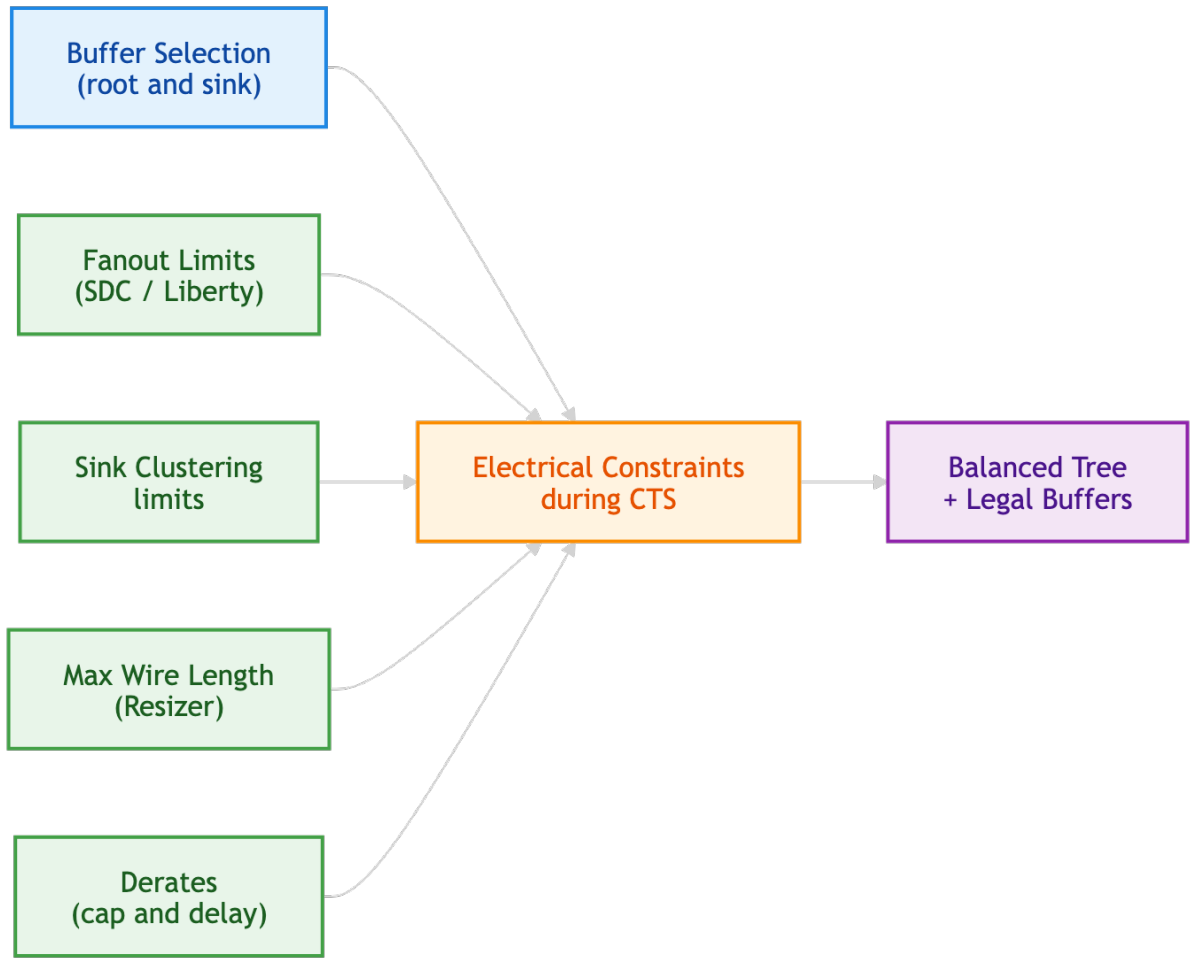
GH-tree (brown) with the downstream clock trees (black) within sink regions



GH-tree (brown) with clustered downstream sinks

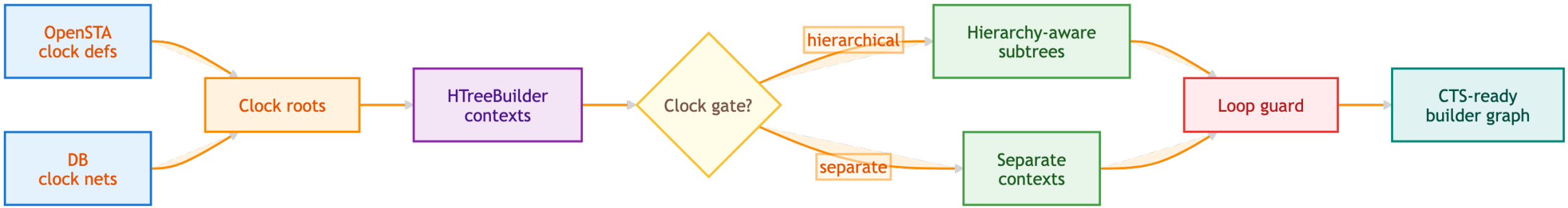
Buffer Selection and Electrical Limits

- Root and sink (LCB) buffer choices finalized during characterization setup
- Fanout limits are read from SDC and Liberty where available
- Sink clustering sizes can be limited by LCB (leaf clock buffer) fanout limits
- Resizer max-wire-length guidance can be used after CTS
- Derates adjust pessimism for sink buffer max cap and delay buffers



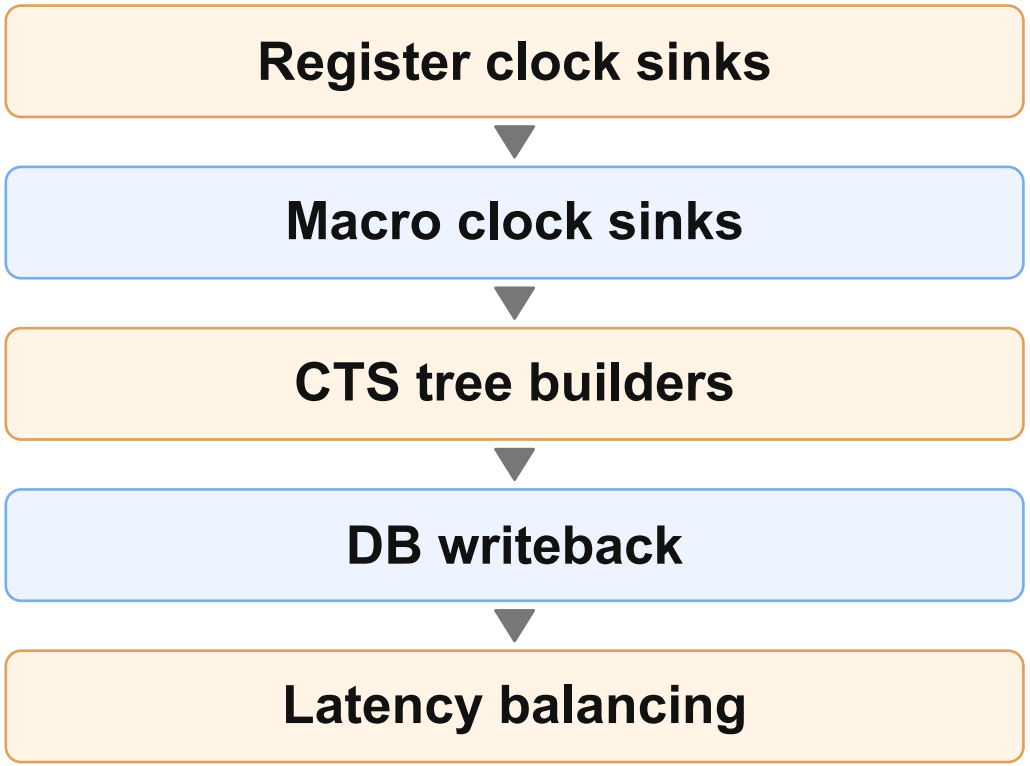
Clock Roots, Gated Clocks, and Builders

- Clock roots found from OpenSTA clock definitions and database nets
- Each valid clock tree gets an `HTreeBuilder`
- Gated clocks create hierarchical or separate clock-tree construction contexts
- Top input nets and parent builders preserve hierarchy relationships
- Recent changes: avoid recursion through clock-gating loops, etc.



Hierarchy and Macro/Register Balancing

- Macro clock pins may have existing internal insertion delay
- Register sinks and macro sinks should have balanced arrival times
- Generated/gated clock trees can form nested builder relationships
- Latency balancing is applied after tree construction/writeback context exists



NDR, Repair, and Electrical Cleanup

- `-apply_ndr` controls whether clock nets get non-default-rule treatment
- `-repair_clock_nets` invokes electrical repair for long/high-fanout clock nets
- `-distance_between_buffers` controls wire segmentation
- `-sink_buffer_max_cap_derate` and `-delay_buffer_derate` tune pessimism
- Post-CTS repair is crucial for final QOR (fmax)

NDR mode	Meaning
none	no clock NDR application
root_only	apply near root only
half	partial clock tree NDR strategy
full	apply broadly through clock tree

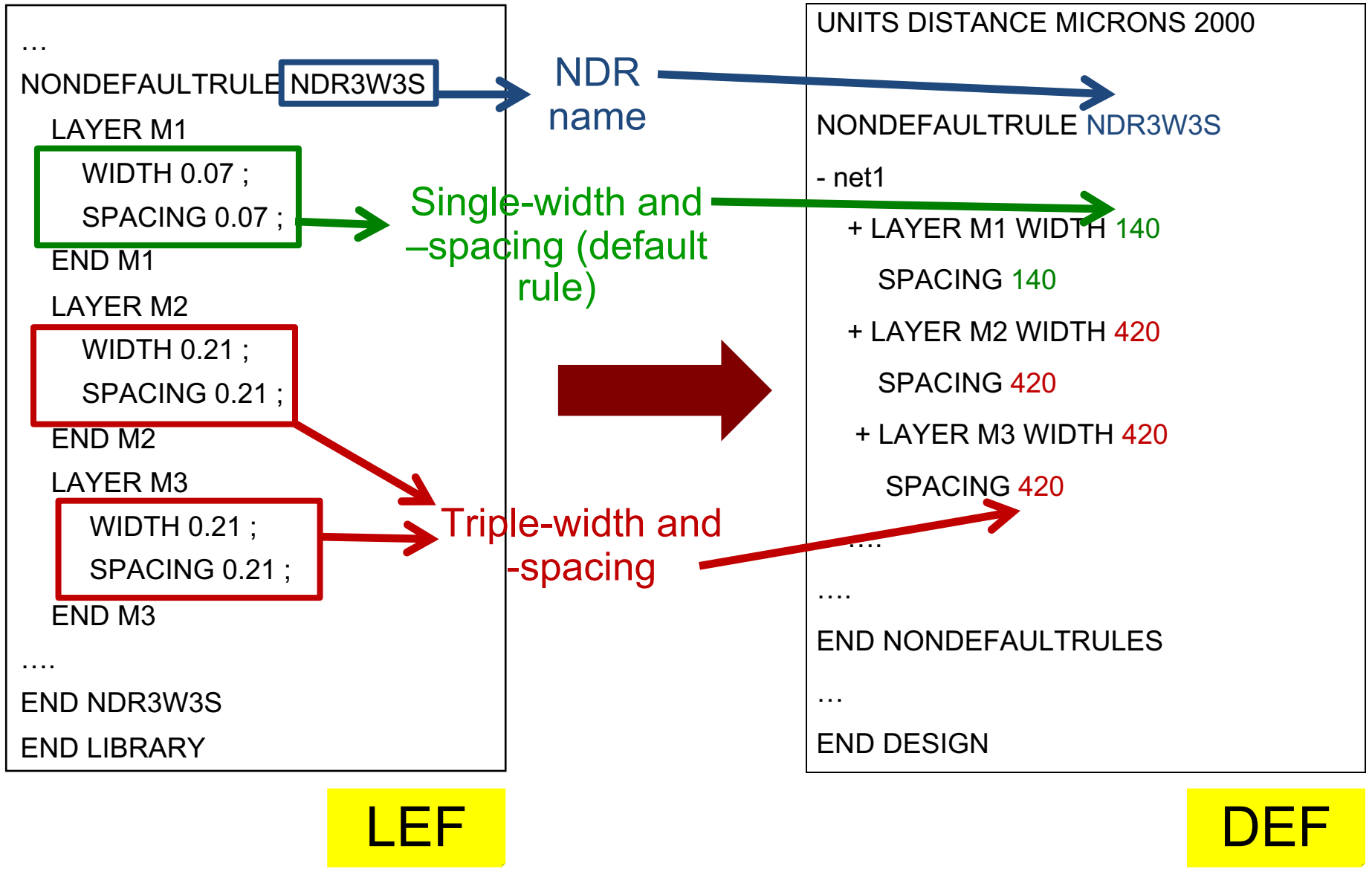
Nondefault Rules (NDRs)

- NDRs apply wider wire widths and spacings that are different from the default rules specified in technology Library Exchange Format (LEF) files as long as they do not violate any design rules (DRC)
  - widening wires decreases sheet resistance ⇒ smaller wire delay
  - NDR examples: double-width and –spacing (2W2S); triple-width and –spacing (3W3S), …
- NDRs are used to fix electromigration (EM) violations, reduce parasitic and delay variations
  - e.g., in clock routing to meet insertion delay/skew requirements
  - e.g., at post-route stage to fix EM violations by widening violating net segments
- Side effects of widening wires with NDRs
  - Increase in driven capacitance ⇒ increase in dynamic power

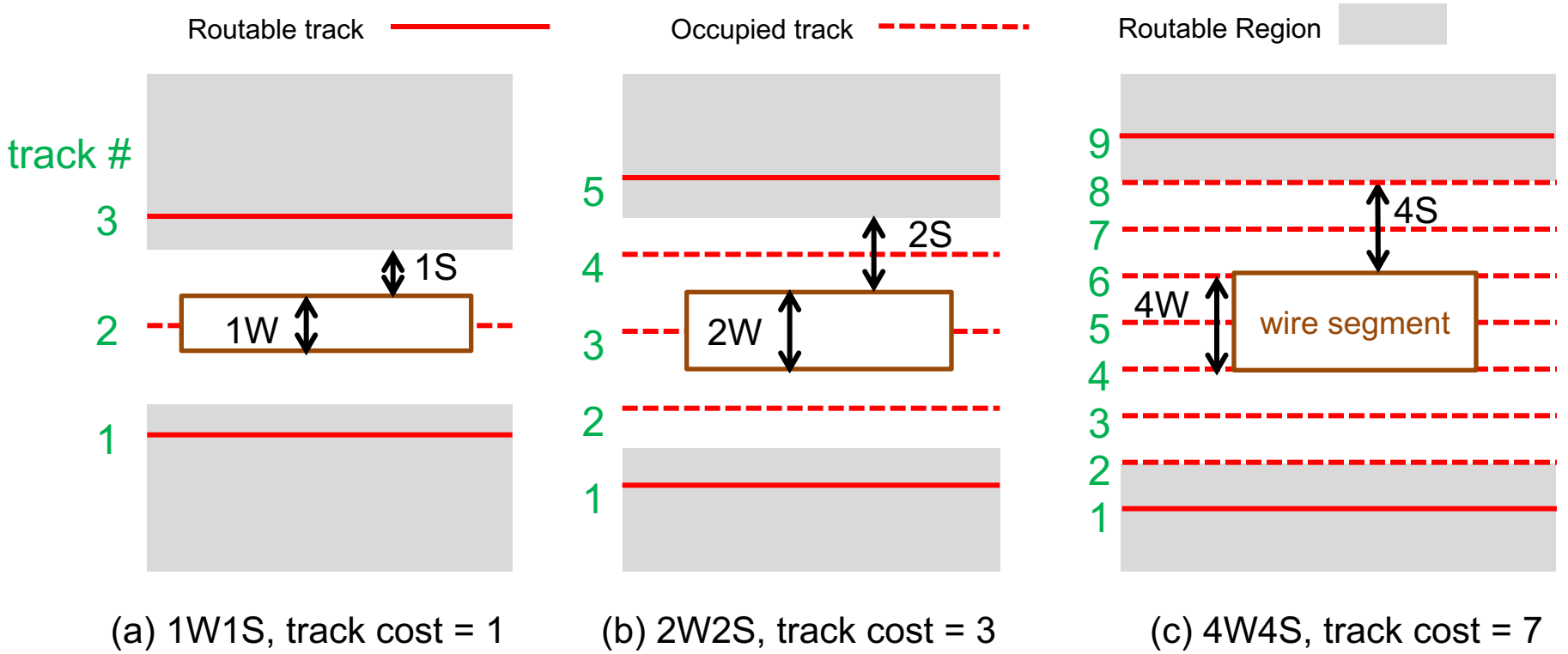
Use of NDRs in Clock Routing

- Eliminate noise
  - Coupling capacitance induces delay uncertainty. Larger spacings can reduce delay uncertainty.
- Prevent EM violation
  - When larger buffers drive minimum-width wires, they overshoot the current density limit. So, wider wires MUST be used to prevent EM violations
- Reduce variation
  - Smaller geometries and more resistive interconnects, in conjunction with multi-patterning in lithography (< 20nm), result in large parasitics and delay variations. Wider wires typically have less variation
- Reduce power (complementary: reduce skew)
  - Wide wires increase capacitance which leads to increase in dynamic power. Applying NDRs smartly (i.e. by wire-tapering), total capacitance (hence power) can be reduced

Specification of NDRs in LEF, DEF



Track Impact of NDRs



(a) 1W1S, track cost = 1 (b) 2W2S, track cost = 3 (c) 4W4S, track cost = 7

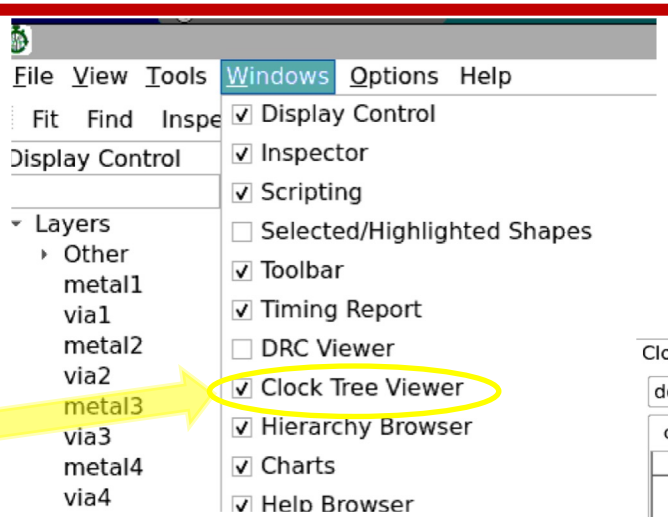
- Track cost = Number of routing tracks unavailable to the detailed router because of increase in the required spacing to neighboring wires when applying NDRs
- Key: Router centers each wire segment on a track gridline

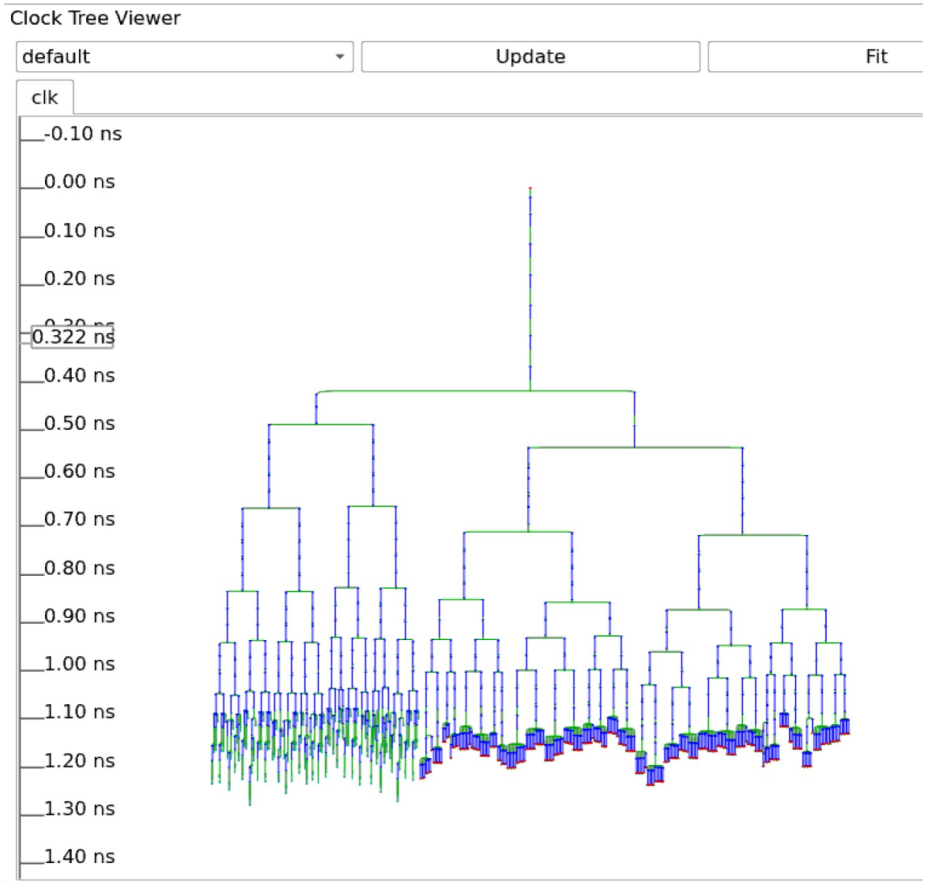
After CTS Construction: Database Writeback

- CTS creates clock buffer instances and connects them to generated subnets.
- Uses stable prefixes: `clkbuf_*`, `clknet_*`, `clkload_*`, `delaybuf_*`
- Can create dummy loads unless disabled
- Applies NDR strategy when requested
- Counts and reports sinks after writing the tree

Clock Tree Viewer

- Open GUI
  - gui::show
- Enable “Clock Tree Viewer” if not enabled
- Clock tree viewer shows latencies at all sinks
  - Red sinks = FF/latches
  - Green sinks = macros
    - Insertion delays are added to macro sinks

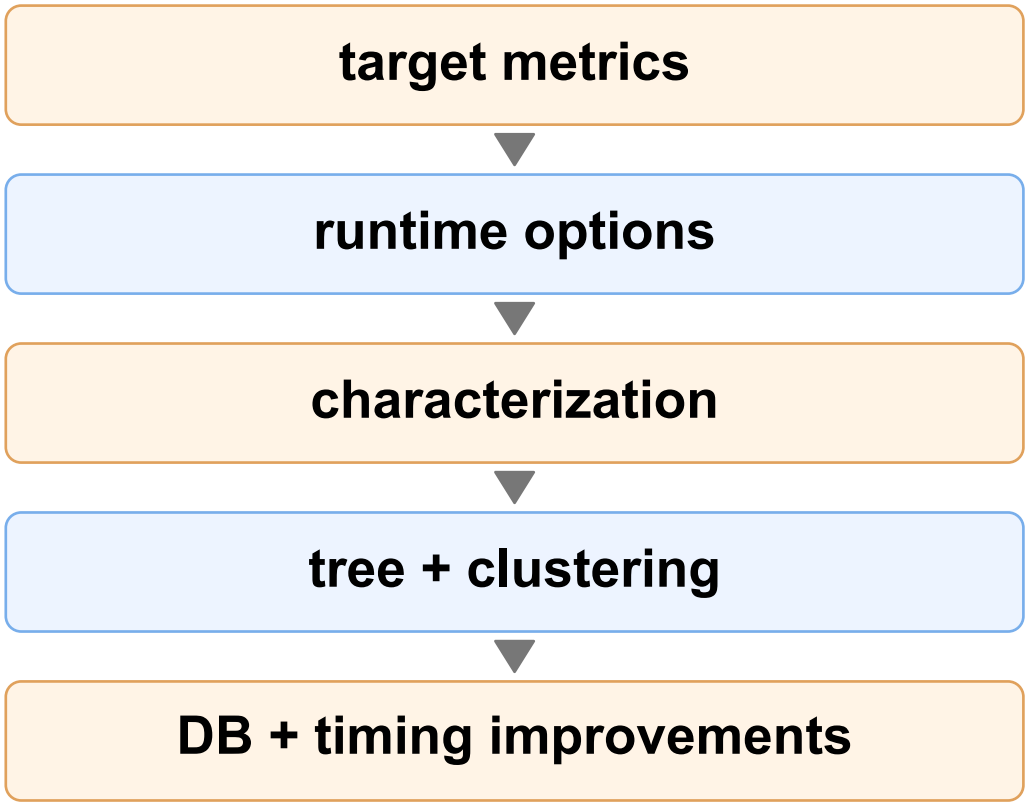




Thanks: Dr. Cho Moon, Precision Innovations

Summary

- OpenROAD CTS is a database-integrated, based clock-tree builder
- Sink clustering and obstruction-aware legalization, “GH-tree” antecedents, “spacefilling-curve clustering” are part of CTS even today.
- Many enhancements over time: simplified characterization, NDR support, sink polarity and hierarchy support, etc.
  - = progression from GH-tree concepts to more robust hierarchical CTS



Watch These Spaces …

- CTS
  - MBFFs (this year's MP1)
  - OCV
  - Clock-data co-opt
  - Useful skew and retiming
- GRT
  - Timing-driven (with budgeting!)
  - RC-aware (beyond 'per-dbu-per-layer' or 'per-via' costs)
  - Better parasitic estimations earlier
  - Empowered to change the placement
  - Blurring with the detailed router
  - GPU acceleration (no easy decomposition into subproblems as in GRT)

BACKUP

On-Chip Variation

- Variations on a chip
  - Process, voltage, and temperature vary in different areas
  - Timing could be optimistic without considering this On-Chip Variation (OCV)

- Traditional OCV
  - Launch paths and capture paths are calculated at different conditions
  - Constant OCV for all paths is pessimistic

    If on-chip delay variation is 2% per mm

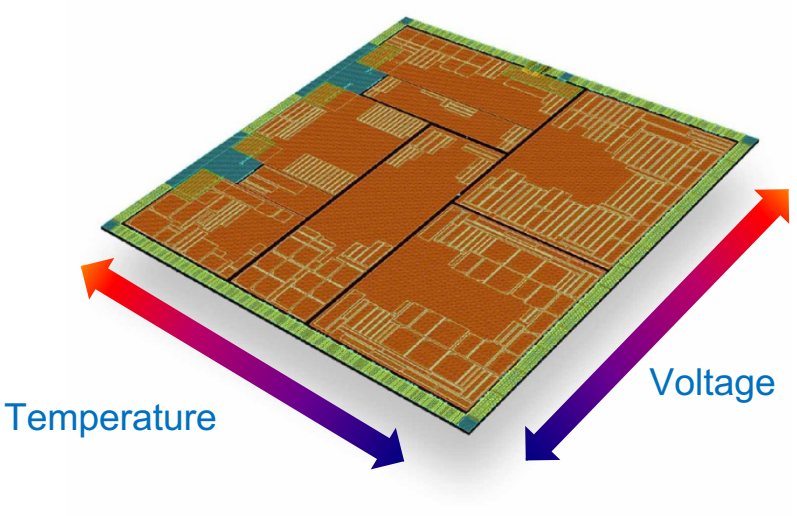
    and the worst path is 10mm

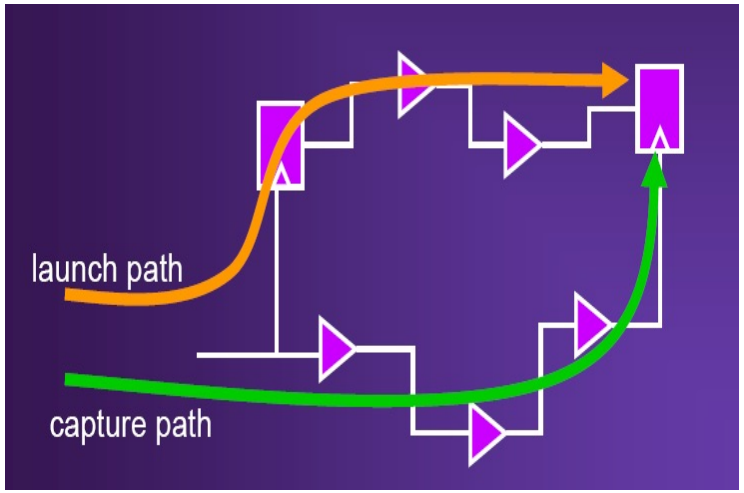
    2% * 10mm = 20%

    20% OCV for all path costs project time!

  - Need more accurate OCV →

    "Location-based" or "Advanced" OCV





Location-based On-Chip Variation

- On-chip variation is fairly smooth, depending on the distance

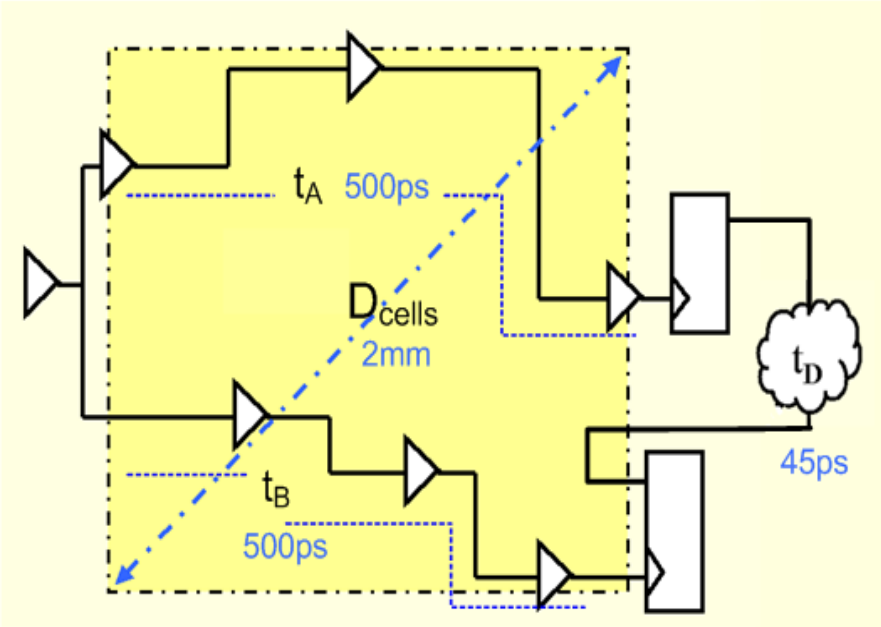
- 2-parameter OCV model [1]

$$Var_{loc} = Var_{mm} * D_{cells} + Var_0$$

Var_{loc} : location-based variation
Var_{mm} : variation per millimeter
D_{cells} : diagonal of cell bounding box
Var_0 : constant variation

- Example : OCV vs. LOCV
  - Hold constraint : $t_A + t_D \geq t_B$
  - OCV case
    t_B is 10% slower
    500 + 45 < 1.10 * 500 : slack -5ps
  - LOCV case
    Var_{mm}: 0.02, Var_0: 0.01
    500 + 45 > 1.05 * 500 : slack 20ps

0.02 * 2 + 0.01 = 0.05



[1] Denis Bzowy, “LOCV : location-based on chip variation” *SNUG Europe*(2004)